

我这个项目全是靠着傻逼愚蠢的豆包写出来的，对我的prompt能力真是锻锻又炼炼啊，需要每次提醒“先plan再execute和我确认沟通好之后再修改代码”，“注意分段读防止超时”，需要把代码解耦模块化不然上下文直接超标截断.....当然不管怎么说，这个我自己的agent在没有经历太多挫折前提下的诞生了。

人形机器人导航智能体

具备记忆检索 → LLM 任务规划 → Skill 执行（延迟参数现场解析 + 技能重试/步间重规划两层纠错） → 记忆写入 → 任务 Review 全流程能力的导航智能体。规划期由关键词小调用与 Planner 主调用两个文本 LLM 协作，执行期延迟步骤再由 Resolver LLM 现场填参，skill 内部由 VLM 完成视觉感知。底层包含三大 Skill 共 7 个可执行功能，支持三种使用方式：全流程自主智能体、手动编排长序列任务、单独运行单个功能。

1 终端命令行

1.1 环境准备

```
scp -P 2222 -r skills/common
root@172.16.11.52:/root/navigation/src/navigation_agent_ultra/skills/
ssh root@172.16.11.52 -p 2222 # 密码naviai@2025
export ROS_MASTER_URI=http://192.168.217.1:11311
export ROS_IP=192.168.217.100
export DASHSCOPE_API_KEY="your-dashscope-key"
export VLM_API_KEY=55842985-94b4-498d-b005-064962f0f894
export KSC_API_KEY=55842985-94b4-498d-b005-064962f0f894
source ~/navigation/devel/setup.bash
cd ~/navigation/src/navigation_agent_ultra
cd ~/navigation/past
```

1.2 使用方式

智能体自主编排

```
python3 agent.py "去找椅子"
python3 agent.py "找到白色桌子上的椰子水并靠近它"
python3 agent.py --file task.txt # 从文件读取任务指令
```

```
from agent import NavigationAgent
agent = NavigationAgent()
result = agent.run("去找椅子")
print(result["success"]) # True/False
print(result["task_understanding"]) # LLM 对任务的理解
print(result["plan"]) # 分步计划
print(result["execution"]) # 执行报告
print(result["review"]) # Review 结果
```

手动编排 Skill 序列 (api详见第三点)

```
# 列出所有技能和功能
python3 main.py list

# 单步调用
python3 main.py call physical_look_around rotate_find_align --target "椅子"
python3 main.py call close_to approach_diagonal --target "货架上的绿色饮料瓶" --
scene-type shelf --direction 0
python3 main.py call execute_action navigate_to_pose --x 2.9 --y -3.8 --yaw 0

# 从 JSON 文件执行长序列任务
python3 main.py run-task --file examples/steps_example.json

# 直接传 JSON 字符串
python3 main.py run-task --json
' [{"skill": "execute_action", "action": "navigate_to_pose", "params": {"x": 1, "y": 1}} ] '

# 某步失败时不停止, 继续执行后续步骤
python3 main.py run-task --file steps.json --no-stop-on-failure
```

```
[
  {
    "skill": "physical_look_around",
    "action": "observe_surroundings",
    "params": {},
    "name": "环境观察"
  },
  {
    "skill": "physical_look_around",
    "action": "rotate_find_align",
    "params": {"target": "椅子"},
    "name": "旋转对正椅子"
  },
  {
    "skill": "close_to",
    "action": "approach_aligned",
    "params": {"target": "椅子", "scene_type": "ground"},
    "name": "逼近椅子"
  }
]
```

```
from main import RobotAgent

agent = RobotAgent()

# 单步调用
result = agent.dispatch("physical_look_around", "rotate_find_align", target="椅子")

# 长序列任务
steps = [
  {"skill": "physical_look_around", "action": "observe_surroundings", "params": {}},
  {"skill": "physical_look_around", "action": "rotate_find_align", "params": {"target": "椅子"}},
  {"skill": "close_to", "action": "approach_aligned", "params": {"target": "椅子", "scene_type": "ground"}}
]
```

```

    {"skill": "physical_look_around", "action": "rotate_find_align",
     "params": {"target": "椰子水"}},
    {"skill": "close_to", "action": "approach_aligned",
     "params": {"target": "椰子水", "scene_type": "shelf"}},
]
report = agent.run_task(steps)

```

2 结构与全流程

2.1 结构

```

navigation_agent_ultra/
├─ agent.py                # 全流程智能体主入口 (NavigationAgent)
├─ main.py                 # 手动编排入口 (RobotAgent: 单步/长序列调度)
├─ task_runner.py          # 长序列任务线性执行器 (无 LLM, 调试/回放用)
├─ llm_client.py           # 文本 LLM 调用封装 (DashScope qwen3.7-plus)
├─ planner.py              # 任务规划器 (关键词提取 + LLM规划 + 步间重规划)
├─ memory_manager.py       # 双层记忆管理 (持久化库 + 工作记忆)
├─ reviewer.py             # 任务级 Review (成败判定 + 失败归因 + 经验教训)
├─ README.md
├─ architecture_overview.html # 架构总览图 (浏览器打开)
├─ logs/                   # 运行日志目录 (运行时自动生成)
├─ tests/                  # 测试脚本 (不需要 ROS)
│   └─ test_motion.py      # 运动参数与工具函数测试
│   └─ test_pid.py         # PID 控制器测试
│   └─ test_planner.py     # 规划器校验逻辑测试
│   └─ test_skills.py      # Skill 注册表与参数校验测试
│   └─ test_vlm.py         # VLM 工具层测试
│   └─ test_memory.py      # 记忆系统功能测试
│   └─ test_replan.py      # 重规划判定逻辑测试
│   └─ test_direction_mapping.py # 方向映射与坐标变换测试
│   └─ test_json.py        # JSON 解析容错测试
├─ memory/                 # 持久化记忆库
│   └─ semantic_map.json   # 物体位置 + 区域信息
│   └─ task_history.json   # 历史任务记录 (含成功/失败/教训)
│   └─ environment_profile.json # 环境常量
│   └─ task_snapshots/     # 单次任务完整快照归档
├─ examples/
│   └─ steps_example.json  # 手动编排长序列任务示例
├─ skills/                 # 三大技能包
│   └─ __init__.py         # 技能注册表 + 统一 dispatch
├─ physical_look_around/   # Skill 1: 环境感知与目标搜索 (4个子功能)
│   └─ __init__.py
│   └─ advance_search_turn.py
│   └─ explore_no_align.py
│   └─ observe_surroundings.py
│   └─ rotate_find_align.py
├─ close_to/               # Skill 2: 目标逼近 (2个子功能)
│   └─ __init__.py
│   └─ approach_aligned.py
│   └─ approach_diagonal.py
├─ execute_action/         # Skill 3: 基础位姿执行 (1个子功能)
│   └─ __init__.py

```

2.2 全流程

用户自然语言指令

- |
- └─ ① 关键词提取 (LLM 小调用)
 - | → {"targets": ["椅子"], "actions": ["找"], "constraints": []}
 - |
- └─ ② 记忆检索 (纯代码, 不用 LLM)
 - | → 从 semantic_map / task_history / environment_profile 中匹配关键词
 - | → 已知高置信度位置优先使用, 历史失败教训辅助避坑
 - |
- └─ ③ 任务规划 (LLM 主调用)
 - | → 系统 Prompt 含三大 skill 详解 + 组合模式 + 记忆规则
 - | → 输出分步计划 JSON (定不下来的参数用 resolve/resolve_hint 延迟)
 - |
- └─ ④ 执行 (延迟解析 + 两层纠错, 详见第 6 节)
 - | → 延迟步骤执行前由 Resolver LLM 结合最新位姿/记忆现场填参
 - | → 依次执行每步 skill, 失败先技能层 retry (同参数重试 1 次)
 - | → 成功: 高置信度感知(≥ 0.7)实时写入 semantic_map
 - | → retry 仍失败: LLM 重规划剩余步骤 (最多 2 次), 替换后继续执行
 - |
- └─ ⑤ 写入记忆
 - | → 工作记忆完整归档到 task_snapshots/
 - | → 高价值信息提炼合并到持久化记忆库
 - |
- └─ ⑥ 任务 Review
 - | → 以最后一步 success 判定成败
 - | → 失败归因 + 生成经验教训写入 task_history
 - | → 下次任务检索时自动参考

3 三大 Skill 概览

3.1 Skill 1: physical_look_around

action	功能	适用场景	前置条件
advance_search_turn	前进找物, 到位后旋转	知道目标大致在前方, 需边前进边搜索	有明确初始朝向
explore_no_align	8方位扫描+信息增益选路+多轮探索, 找到后旋转回发现方位即停	完全不知道目标在哪, 需自主搜索; 找到后紧接 approach	无
observe_surroundings	8方向拍照+VLM分析, 构建语义记忆	任务开始时了解环境	无

action	功能	适用场景	前置条件
<code>rotate_find_align</code>	粗搜索+精对正，使目标在视野中央	知道目标大致方向范围	目标在可旋转范围内

advance_search_turn.py

前进找物 + 到位旋转：沿当前方向前进，每 2m 停车 VLM 找物，找到且深度 < 3m 时停止，然后旋转到指定绝对角度。

```
python3 skills/physical_look_around/advance_search_turn.py \
  --target "椅子" \           # 必填，目标物体描述
  --angle 90 \                 # 必填，停止后旋转到的绝对角度（度，世界系）
  --camera chest \            # 可选，摄像头 head/chest，默认 chest
  --scene-type ground \       # 可选，场景 shelf/ground，默认 ground
  --step-dist 2.0 \           # 可选，每步前进距离(m)，默认 2.0
  --stop-depth 3.0 \          # 可选，物体深度小于此值时停止(m)，默认 3.0
  --max-dist 20.0             # 可选，最大前进距离安全限制(m)，默认 20.0
```

explore_no_align.py

自主探索（找到后旋转回发现方位即停）：原地 8 方位旋转扫描，VLM 分析开阔度和物体，选择信息增益最大的方向前进，多轮迭代，找到目标后旋转回发现方位即停（不做最终对齐）。适合后续紧接 approach 的场景。

```
python3 skills/physical_look_around/explore_no_align.py \
  --target "瓶子" \           # 可选，目标描述，默认使用内置默认目标
  --max-rounds 10 \           # 可选，最大探索轮数，默认 10
  --camera chest \            # 可选，head/chest，默认 chest
  --out-dir explore_logs \     # 可选，数据输出目录，默认 explore_logs
  --exclude-highest 2          # 可选，每轮排除开阔度最高方位数(不含最高)，默认 2
```

observe_surroundings.py

8 方向环境观察：原地旋转 8 方向（0°/45°/.../315°），每方向拍照 + VLM 分析场景，构建语义记忆。不移动、不搜索特定目标。

```
python3 skills/physical_look_around/observe_surroundings.py \
  --camera chest \            # 可选，head/chest，默认 chest
  --out-dir observation_logs \ # 可选，数据输出目录，默认 observation_logs
  --no-return                 # 可选，观察后不转回初始朝向（不加默认转回）
```

rotate_find_align.py

原地旋转找物并对正：原地旋转，粗搜索（0°/90°/180°/270° + 45°/135°/225°/315°）找到目标，然后精对正（像素偏移算角度）使目标在视野中央。

```
python3 skills/physical_look_around/rotate_find_align.py \
--target "椅子" \           # 必填，目标物体描述
--camera chest \           # 可选，head/chest，默认 chest
--scene-type ground \      # 可选，shelf/ground，默认 ground
--tolerance-deg 3.0        # 可选，中央对正容差角度(度)，默认 3.0
```

3.2 Skill 2: close_to

action	功能	适用场景	前置条件
<code>approach_aligned</code>	VLM计算偏移，导航到目标前0.5m	rotate_find_align 之后，目标已正对	目标在视野中且已正对
<code>approach_diagonal</code>	沿cardinal方向前进后转90°对正	目标斜前方，有障碍不能直线逼近	目标在视野中，有可前进方向

approach_aligned.py

正对目标逼近：视野中已存在目标且已正对时，拍照 + VLM 计算左右偏移和前后距离，导航到目标附近（距支撑面 0.5m）。

```
python3 skills/close_to/approach_aligned.py \
--target "椰子水" \       # 必填，目标物体描述
--camera chest \          # 可选，head/chest，默认 chest
--scene-type shelf        # 可选，shelf/ground，默认 shelf
```

approach_diagonal.py

斜角逼近：视野中存在目标但斜对时，沿指定 cardinal 方向（0°/90°/180°/270°）前进，然后转 90° 对正目标。

```
python3 skills/close_to/approach_diagonal.py \
--target "椅子" \         # 必填，目标物体描述
--direction 0 \           # 必填，可前进方向 0/90/180/270
--camera chest \          # 可选，head/chest，默认 chest
--scene-type ground       # 可选，shelf/ground，默认 ground
```

3.3 Skill 3: execute_action

action	功能	适用场景	前置条件
<code>navigate_to_pose</code>	导航到指定(x,y,yaw)	已知目标精确坐标	知道精确坐标和朝向

navigate_to_pose.py

导航到指定位姿：导航到指定坐标(x,y,yaw)，等待机器人到达后返回。纯运动无感知。底层

`send_navigation_goal` 既是独立 Skill 的执行函数，也被其他 Skill 内部作为辅助运动原语引用。

```
python3 skills/execute_action/navigate_to_pose.py \
--x 2.9 \           # 必填，目标 x 坐标(m)
--y -3.8 \          # 必填，目标 y 坐标(m)
--z 0.0 \           # 可选，目标 z 坐标(m)，默认 0
--yaw 0.0 \         # 必填，目标朝向(度)
--task-type 0       # 可选，整数任务类型，默认 0（命令行不校验取值范围）
```

4 双层记忆系统

4.1 总览与全流程时序

两层记忆对比

	持久化记忆库	工作记忆
载体	memory/ 下三个 JSON 文件	内存对象（过程中不落盘，结束才归档）
生命周期	跨任务长期累积	单次任务，任务结束即归档
内容	物体位置、区域、历史任务、环境备注	本次计划、每步结果、逐步感知、发现物体、重规划次数
读取时机	任务开始检索；每次规划/重规划/延迟解析取位置清单	执行过程中实时累积，重规划时作为上下文
写入时机	高置信度发现实时写 + 任务末统一合并	每步执行后追加
磁盘文件	semantic_map / task_history（agent 自动写）、environment_profile（只读，人工维护）	task_snapshots/<task_id>.json

记忆在六步全流程中的时序

全流程步骤	记忆操作
① 关键词提取后	search(keywords) 代码检索持久化库，得到文本上下文
② 任务规划前	汇总全量已知位置清单（持久化库 + 工作记忆估算），连同检索上下文一起喂给 Planner

全流程步骤	记忆操作
③ 规划完成后	<code>init_work_memory()</code> 创建本次工作记忆，并存入 plan
④ 每步执行后	<code>add_step_result()</code> 记录该步结果与感知；置信度 ≥ 0.7 的发现经 <code>update_semantic_map_realtime()</code> 实时写库；重规划时 <code>increment_replans()</code> 并把工作记忆带入 replan
⑤ 执行结束	<code>save_snapshot()</code> 归档完整工作记忆； <code>merge_to_persistent()</code> 把全部感知合并进持久化库
⑥ Review 后	<code>update_task_history()</code> 追加一条历史任务记录（含成败、归因、教训）

文件写入均采用"写临时文件 + 原子替换"，中途异常不会写坏 JSON；文件缺失或损坏时自动回退到空的默认结构。

4.2 持久化记忆库

位于 `memory/` 目录，共三个文件，分工如下（检索细节见 4.4，写入细节见 4.5）：

文件	内容	被谁读取	被谁写入
<code>semantic_map.json</code>	物体坐标/朝向/置信度 + 各方向区域	通道 A <code>search()</code> （按关键词筛物体）、通道 B 全量位置清单	执行中实时写（置信度 ≥ 0.7 ）+ 任务末统一合并
<code>task_history.json</code>	历次任务的成败、归因与教训	仅通道 A <code>search()</code> （匹配指令/教训/原因，最多 5 条）	仅任务 Review 后 <code>update_task_history()</code> 追加一条
<code>environment_profile.json</code>	环境类型与长期备注	仅通道 A <code>search()</code> （notes 非空就原样附加）	智能体不写入，由人工维护

(1) semantic_map.json — 物体位置与区域信息（空间记忆：通道 A、B 都会读；执行中实时写 + 任务末合并）

```
{
  "objects": [
    {
      "name": "椅子",
      "position": {"x": 2.5, "y": -1.0, "theta": 90.0},
      "confidence": 0.85,
      "depth": 2.5,
```



```

    "direction": 90,
    "first_seen": "2026-08-26 10:00:00",
    "last_seen": "2026-08-26 10:30:00",
    "seen_count": 3,
    "source_action": "explore_no_align",
    "status": "confirmed"
  }
],
"areas": [
  {"direction": 0, "openness": 0.8, "description": "前方开阔", "last_seen": "..."}
],
"last_updated": "..."
}

```

objects 中每个物体的字段含义：

字段	含义
name	物体名称，是检索与合并的主键
position	全局坐标 {x, y, theta}; 允许没有精确坐标 ，此时为 {x: null, y: null, theta: 方向}，只保留 depth/direction
confidence	置信度 0~1
depth / direction	发现时的距离(m)与相对朝向(度)
first_seen / last_seen	首次 / 最近一次发现时间
seen_count	累计观测次数，每次合并 +1，同时作为位置加权平均的权重依据
source_action	最近一次由哪个 action 发现
status	物体状态，合并后置为 confirmed

areas 记录各方向开阔度，元素为 {direction, openness, description, last_seen}; 方向差 < 5° 视为同一区域并更新。

(2) task_history.json — 历史任务记录（经验记忆：仅通道 A 读取；任务 Review 后追加一条）

```

{
  "tasks": [
    {
      "task_id": "task_20260826_100000",
      "instruction": "去找椅子",
      "timestamp": "2026-08-26 10:00:00",
      "success": true,
      "steps_planned": 3,
      "steps_completed": 3,
      "replans": 0,
      "failure_reason": null,
      "root_cause": null,
      "lesson": null,
      "objects_found": ["椅子"],
    }
  ]
}

```

```
        "duration_sec": 45.2
    }
]
}
```

字段含义：`success` 成败；`steps_planned/steps_completed` 计划与完成步数；`replans` 重规划次数；`failure_reason/root_cause/lesson` 为失败原因、根本归因、经验教训；`objects_found` 为发现物体名称；`duration_sec` 为耗时。检索时相关历史任务最多取最近 5 条。

(3) environment_profile.json — 环境常量与备注（静态先验：仅通道 A 读取；智能体不写入，人工维护）

```
{
  "environment": "indoor",
  "notes": "货架区在房间北侧，x>4 一侧地面湿滑"
}
```

- `environment`：环境类型标识，默认 `indoor`；
- `notes`：自由文本环境备注，**只要非空，每次 `search()` 都会原样注入 Planner 上下文**，用于记录长期不变的环境先验。

4.3 工作记忆

`init_work_memory()` 在规划完成后创建，整体结构如下：

字段	含义	写入方法
<code>task_id / instruction</code> <code>/ start_time /</code> <code>end_time</code>	任务标识、原始指令、起止时间	init 时写入，归档时补 end_time
<code>plan</code>	本次 LLM 规划结果（任务理解 + 分步）	<code>init_work_memory</code>
<code>steps</code>	每步执行记录： step/skill/action/params/success/message/elapsed_sec	<code>add_step_result</code> 每步追加
<code>perceptions</code>	每步感知快照：step/action + objects_found/areas_explored/final_pose	<code>add_step_result</code> 每步追加
<code>objects_discovered</code>	本次任务发现物体的累积列表	<code>add_step_result</code>
<code>context</code>	action 回传的额外上下文（dict 增量合并）	<code>add_step_result</code>
<code>replans</code>	本任务重规划次数	<code>increment_replans</code>

任务结束时 `save_snapshot()` 把整个工作记忆**原样、不可变地**归档到 `memory/task_snapshots/<task_id>.json`，结构示例：

```
{
  "task_id": "task_20260829_100000",
  "instruction": "找到白色桌子上的椰子水并靠近它",
  "start_time": "2026-08-29 10:00:00",
  "end_time": "2026-08-29 10:01:12",
  "plan": {"task_understanding": "...", "steps": []},
  "steps": [
    {"step": 1, "skill": "physical_look_around", "action":
"observe_surroundings",
```

```
    "params": {}, "success": true, "message": "观察完成：8 个方位", "elapsed_sec":
42.3}
  ],
  "perceptions": [
    {"step": 1, "action": "observe_surroundings",
      "objects_found": [], "areas_explored": [],
      "final_pose": {"x": 0.0, "y": 0.0, "theta": 0.0}}
  ],
  "objects_discovered": [],
  "context": {},
  "replans": 0
}
```

快照只用于事后回溯，不会被后续任务反向读取；跨任务的信息传递只通过 4.5 的合并进入持久化记忆库。

4.4 两套检索通道

智能体对持久化记忆有两条不同的读取通道，分别服务于“任务开始的粗筛”和“执行时刻的精确取参”。通道 A 主要服务第 5 节任务开始检索；通道 B 在第 5 节主规划、第 6 节延迟取参与重规划时反复重新生成。

通道 A：memory.search(keywords) — 代码子串检索（任务开始一次）

- 输入是关键词小调用提取的 {targets, actions, constraints}；
- 物体匹配 name：关键词是物体名子串、或物体名是关键词子串都算命中（双向子串匹配）；
- 历史任务匹配 instruction / lesson / failure_reason / root_cause 四个字段，命中即收录，最多取最近 5 条，失败任务附带教训与失败原因；
- environment_profile.notes 非空则附加在末尾；
- 全部无命中时返回固定文案“未找到与当前任务相关的历史记忆”；
- 输出是格式化文本，作为规划参考注入 Planner。

通道 B：全量已知位置清单 — 供 Planner / Resolver 语义匹配

- get_known_positions_text() 列出持久化库中**所有有坐标**的物体（无坐标物体不列），不做关键词过滤；
- agent 再叠加本次工作记忆的新发现：已有全局坐标的直接列出；只有 direction+depth 的，用该步 final_pose 当场估算坐标（公式见 4.5）；
- 这份清单在主规划、每次重规划、每个延迟步骤 Resolver 取参时都会重新生成，因此包含执行过程中刚刚新发现的物体；
- Resolver LLM 在这份全量清单上做语义匹配（同义词、指代词、上下文），输出结构化参数直接填进待执行步骤。

两通道对比

	通道 A search()	通道 B 全量位置清单
触发时机	任务开始一次	主规划 / 每次重规划 / 每个延迟步骤执行前
输入	结构化关键词	无需关键词，全量列出
记忆范围	按关键词子串过滤后的子集	全部有坐标物体 + 本次任务新发现

	通道 A search()	通道 B 全量位置清单
匹配方式	代码子串, 不能模糊/同义匹配	交由 LLM 语义匹配, 可识别同义词与指代
输出	格式化文本, 给 Planner 作参考	紧凑位置清单, 供 LLM 选出精确坐标
时效性	任务开始时的快照	执行时刻最新

注意: 检索层**不按置信度过滤**, 物体的现有置信度会原样列出供模型判断。"位置置信度 ≥ 0.7 时优先直接 `navigate_to_pose`、 < 0.7 仍需 `explore_no_align/rotate_find_align` 现场确认"是 Planner 的 **prompt 决策规则**, 不是记忆层的过滤逻辑; 历史失败教训同样经通道 A 注入 prompt 以规避已知问题。

4.5 记忆写入与合并规则

两条写入路径

- **实时写入**: 每步执行后 `update_semantic_map_realtime()` 只把 $\text{confidence} \geq 0.7$ 的物体立即合并落盘 (区域信息不受阈值限制, 全部更新), 保证后续步骤立即可用;
- **任务末合并**: `merge_to_persistent()` 遍历工作记忆中的**全部**感知物体 (**包含置信度低于 0.7 的弱观测**) 再合并一次, 避免漏记; 随后归档快照、追加历史任务记录。

物体合并规则 (同名前提下分情况)

已有物体	新观测	处理
有坐标	有坐标, 且距离 $< 1.5\text{m}$	合并为同一物体
有坐标	有坐标, 但距离 $\geq 1.5\text{m}$	不合并, 作为新物体追加
无坐标	无坐标	同名即合并, 避免无位置物体重复堆积
有坐标	无坐标 (或反过来)	不合并 , 视为两次不同观测分别保留

合并为同一物体时:

- `seen_count` +1, 刷新 `last_seen`, `status` 置为 `confirmed`;
- `confidence` 取两者较大值;
- 位置加权平均: 新观测权重 $w_{\text{new}} = 1/\text{seen_count}$ 、旧位置权重 $1 - w_{\text{new}}$, 对 x 、 y 加权平均; `theta` 直接取新观测值;
- `depth`、`source_action` 用新观测覆盖。

无坐标时的位置估算: 新观测没有显式 `position`、但带有 `depth` 和 `direction` 时, 用发现时刻的机器人位姿估算全局坐标:

```
x = 机器人x + depth · cos(direction)
y = 机器人y + depth · sin(direction)
theta = direction
```

若 `depth` 也缺失则不估算, 保留 `{x: null, y: null}` 空坐标。

区域合并: 方向差 $< 5^\circ$ 视为同一区域, 更新 `openness/description/last_seen`, 否则追加。

历史任务追加：任务 Review 后由 `update_task_history()` 追加一条记录（字段见 4.2），其成败与教训会在下一次任务经通道 A 被检索参考。

两个相关常量：物体合并距离 `OBJECT_MERGE_DISTANCE = 1.5`（米），实时写库置信度阈值 `REALTIME_CONFIDENCE_THRESHOLD = 0.7`。

5 长序列任务的理解、检索与拆分

任务开始阶段是一条三步流水线：**关键词提取（LLM 小调用）** → **记忆检索（纯代码）** → **任务规划（LLM 主调用）**。

核心设计是**第一轮 LLM 并不把整个记忆库都塞进去**：先用一次极廉价的小调用把指令抽成结构化关键词，用纯代码从记忆库粗筛出相关子集；主规划只读这份子集，外加一份全量已知位置清单，从而控制 token、避免无关记忆干扰。

用户指令

```
|
| 5.1 extract_keywords (LLM 小调用, max_tokens=256, temperature=0)
|     → {targets, actions, constraints}
|
| 5.2 memory.search (纯代码, 通道 A, 零 LLM)
|     → memory_context: 命中的物体 / 历史任务 / 环境备注
|     同时 agent 另建通道 B 全量位置清单 known_positions (见 4.4)
|
| 5.3 Planner.plan (LLM 主调用, max_tokens=2048, temperature=0.3)
|     system: 当前位姿 + known_positions + 三大 Skill 能力/组合模式/记忆规则
|     user  : memory_context + 用户原始指令
|     → {task_understanding, memory_used, steps[]}
```

5.1 extract_keywords — LLM 小调用

- **时机与次数：**任务开始、主规划之前，恰好一次。
- **输入输出：**输入用户原始指令，输出严格 JSON：`targets`（目标物体名）、`actions`（动作类型）、`constraints`（颜色/位置/数量等约束）。
- **调用参数：**`max_tokens=256, temperature=0`，system 角色固定为"信息提取助手，只输出 JSON"，保证抽取稳定可复现。
- **两级容错，绝不因抽取失败中断任务：**
 1. LLM 无返回 / JSON 无法解析 → `_fallback_keywords` 用内置 18 个常见物体词（椅子、桌子、瓶子、椰子水、饮料等）做子串规则兜底，actions/constraints 置空；
 2. agent 层再包一层 try，任何残留异常都退化为空关键词 `{targets:[], actions:[], constraints:[]}`，继续走后续流程。
- 返回前用 `setdefault` 保证三个字段齐全。**为什么先做这一小调用：**它产出的结构化关键词正是 5.2 纯代码检索的查询输入，使主规划无需面对原始、冗长的全量记忆。

5.2 memory.search(keywords) — 纯代码检索 (通道 A)

- **零 LLM、零额外 token**，完全由代码子串匹配完成，规则细节见 4.4 通道 A。
- 输出一段格式化文本 `memory_context`：命中的已知物体位置、相关历史任务（最多最近 5 条，失败任务带教训）、环境备注；全部无命中时返回固定文案"未找到与当前任务相关的历史记忆"，由 Planner 自行决定搜索策略。
- 与此同时，agent 调用 `_build_positions_text()` 另建**通道 B 全量位置清单** `known_positions`（持久化库有坐标物体 + 本次工作记忆估算位置，不做关键词过滤，见 4.4）。
- **两份材料去向不同**：`memory_context` 进入主调用的 user 消息作为"参考经验"；`known_positions` 与当前位姿一起填入 system prompt，作为"可直接使用的坐标"。

检索走例：三个 JSON 各自贡献什么

以指令"找到白色桌子上的椰子水并靠近它"为例：

1. 5.1 小调用提取出关键词，如 `targets:["桌子","椰子水","饮料瓶"]`；
2. `search()` (通道 A) 用这些关键词**同时查三个持久化文件**：
 - **semantic_map.json**：在 objects 的名字上做子串匹配——"桌子"命中名为"白色桌子"的物体、"椰子水"命中"椰子水饮料瓶"，输出它们的坐标、置信度、最后发现时间；
 - **task_history.json**：在每条任务的 instruction/lesson/failure_reason/root_cause 上匹配，命中上一次"找椰子水"的记录并附上其教训（最多取最近 5 条）；
 - **environment_profile.json**：notes 非空（如"货架区在房间北侧"）就原样附在检索结果末尾；三者拼成 `memory_context`；若三个文件都无命中，则返回"未找到与当前任务相关的历史记忆"。
3. 与检索并行，**通道 B 只从 semantic_map.json 取所有带 x/y 的物体**（不查另外两个文件），再叠加本次工作记忆的新发现，形成 `known_positions` 清单；
4. 5.3 主规划时，来自三个文件的 `memory_context` 进 user 消息，只来自 semantic_map 的 `known_positions` 进 system prompt，Planner 据此判断"已知高置信位置直接 navigate，还是先搜索确认"。

一句话记忆：**通道 A 会翻遍三个 JSON，通道 B 只用 semantic_map.json 里的有坐标物体。**

5.3 Planner.plan — LLM 主调用

- **时机与次数**：任务开始一次，`max_tokens=2048`，`temperature=0.3`。
- **Prompt 构成**：
 - **system (SYSTEM_PROMPT)**：填入当前位姿（`x/y/朝向`，取不到时显示"未知"）与通道 B 清单（为空显示"（无）"）；正文包含三大 Skill 共 7 个 action 的能力详解（适用场景/前置条件/参数必填与默认值）、延迟参数说明、三条 Skill 组合链（标准搜索链 / 未知区域探索链 / 已知位置直达链）、记忆使用规则（置信度 > 0.7 优先 navigate 直达）、严格 JSON 输出格式；
 - **user**：**【记忆检索结果】**`memory_context` + **【任务指令】**原始指令。
- **输出 JSON 三字段**：
 - `task_understanding`：一句话任务理解；
 - `memory_used`：本次用到了哪些记忆条目（无则空数组）；

- `steps[]`：分步计划，每步含 `step`（自动重编为 1..N）、`skill`、`action`、`params`、`rationale`（选这步的理由），以及可选的 `resolve/resolve_hint`（延迟参数，见 6.1）。
- 规划后四道校验，任一不过即抛错、任务走 `_abort`：
 1. `steps` 必须存在、为非空列表；
 2. 每步必须有 `skill` 和 `action`，并自动补齐 `params/rationale/resolve` 默认值、重排序号；
 3. `_validate_steps` 对照 Skill 注册表校验 `skill/action` 是否属于 7 个合法 action，错误信息带步骤号；
 4. `_validate_resolve_fields` 校验延迟字段：`resolve` 必须是列表、有 `resolve` 必须带 `resolve_hint`、被 `resolve` 的参数不能同时出现在 `params` 中。
- LLM 返回无法解析为 JSON 时直接抛 `RuntimeError("LLM 规划失败")`。通过校验的 plan 会整体存入工作记忆，`steps` 交给第 6 节的执行/纠错循环。

6 延迟传参与纠错（repeat / replan）

计划进入执行后，由两套机制保证长序列鲁棒性：

- **延迟传参**：规划时刻定不下来的参数，推迟到执行前由 Resolver LLM 结合最新位姿与记忆现场确认；
- **两层纠错**：技能层 **repeat**（同参数重试，消瞬时故障）与编排层 **replan**（LLM 重写剩余步骤，换策略）。

6.1 延迟传参机制（Resolver LLM）

解决什么问题：有些参数在规划时刻根本不存在，硬填只能靠 LLM 编造，因此 Planner 只写"占位声明"，把取值推迟到执行现场。两类典型延迟参数：

延迟参数	所属 action	为什么要延迟	解析方式
x / y / yaw	navigate_to_pose	目标坐标要等走到附近、记忆里有了才知道（如"第二个巡检点"）	从通道 B 全量位置清单中语义匹配坐标
angle	advance_search_turn	只知道"左转/右转/掉头"，绝对角度取决于执行时刻的当前朝向	按当前朝向换算：左转 +90、右转 -90、掉头 +180，结果归一化到 0-360

step 字段约定：

- `resolve`：字符串数组，列出要延迟确定的参数名；
- `resolve_hint`：自然语言，写清"找什么、有什么特征"或"相对转向方向与度数"；
- 被 `resolve` 的参数**不得**同时出现在 `params` 中；规划时就能确定的参数直接填 `params`，不滥用延迟；不需要延迟时这两个字段都不写。

执行前解析流程（每个延迟 step、dispatch 之前）：

1. 先取机器人当前位姿；
2. 仅当该 step 带 **resolve** 才触发，无 `resolve` 的步骤零 LLM 调用，直接用原 `params`；

- 3. 重新构建通道 B 全量位置清单（因此包含前面步骤刚刚发现、实时写入的物体）；
- 4. 调用 Resolver（`max_tokens=256, temperature=0`），输入当前位姿、action 名、待解析参数列表、`resolve_hint`、全量位置清单，输出 `{found, params, reason}`；
- 5. 对返回值做严格校验后，把解析结果合并进 `params` 并回写 `step.params`，再进入 `dispatch`。

Resolver 校验链（任一不过抛 `ResolutionError`）：LLM 无返回；`found=false`；`params` 不是字典；缺少任一声明的参数；值无法转为数值；数值非有限数；`x/y` 超出 $\pm 100m$ ；`angle/yaw/theta/heading` 类归一化到 $[0,360)$ ；其他参数超出 ± 1000 。通过后数值保留 3 位小数。

与 4.4 呼应：通道 A 是代码子串、不能同义匹配；Resolver 基于**通道 B 全量清单**做 **LLM 语义匹配**，能理解同义词与指代词（"第二个任务点"→"巡检点 B"）。`ResolutionError` 属于**确定性失败**，**不做技能层重试，直接进入 6.4 重规划**。

走例 1（查坐标）：计划步为 `{action:"navigate_to_pose", params:{}, resolve:["x","y","yaw"], resolve_hint:"第二个巡检点的坐标与朝向"}`。执行前位姿为 $(1.2, 0.5, 90^\circ)$ ，通道 B 清单里列有巡检点 A/B；Resolver 语义匹配到"巡检点 B"，返回 `{found:true, params:{x:7.1, y:2.3, yaw:180}}`，校验通过后 `params` 被填为 `{x:7.1, y:2.3, yaw:180.0}` 再导航。

走例 2（算角度）：当前朝向 0° ，计划步 `resolve:["angle"]`、`hint` 为"右转 90° ，换算为绝对角度"，Resolver 返回 `angle=270`（ $0-90$ 归一化），`advance_search_turn` 到位后旋转到 270° 。

6.2 两层纠错总览



	技能层 repeat	编排层 replan
所在层	单个 action 内部	任务编排层
是否调 LLM	否，同参数重发	是，重新规划剩余步骤
目的	消除瞬时故障（VLM 偶发漏检、旋转差几度、导航瞬时报错）	原策略走不通时换策略
上限	<code>SKILL_MAX_RETRIES = 1</code> （共尝试 2 次）	<code>MAX_REPLANS = 2</code>
触发后	参数不变、重新 dispatch	用新 steps 替换失败步及其后所有步骤

相关常量：`SKILL_MAX_RETRIES=1`、重试间隔 `RETRY_WAIT_SEC=1.0` 秒、`MAX_REPLANS=2`。

6.3 技能层 repeat (action 单次失败的同参数重试)

- **触发**: dispatch 抛异常 (被包装成 `success=False` 的失败结果), 或 skill 返回 `success=False`。
- **不重试**: 延迟解析失败、位姿获取失败这类**确定性错误**——参数本身就不对, 重试无意义, 直接交给 replan。
- **重试行为**: 先 `sleep(1.0s)` 等待瞬态条件消除; 重试前**重新读取当前位姿** (机器人失败后可能有微小位移); 用**同一批已解析好的 params 重新 dispatch**, **不重新调用 Resolver**。首次 + 重试共 `SKILL_MAX_RETRIES+1 = 2` 次尝试, 任一尝试成功即停止。
- **记录口径**: 每一次尝试 (含失败尝试) 都写入工作记忆便于事后追溯; 但执行报告 `results` 中每个逻辑步骤**只保留最终一次结果**, 发生过重试时附带 `retries` 字段, 重试次数同时累计到全局 `skill_retries`。

走例: 执行 `close_to.approach_diagonal` 逼近"货架上的绿色饮料瓶", 第 1 次返回 `success=False`、message 为"逼近未完全成功 (最终旋转误差 8.2°)"。判定为瞬时故障 → 等 1.0s、重读位姿、用相同参数再发一次; 第 2 次旋转到位、返回成功。于是该步在 `results` 中只留一条成功记录并标注 `retries:1`, 全局 `skill_retries=1`, **不消耗 replan 次数**, 继续下一步。

6.4 编排层 replan (剩余步骤重规划)

- **触发**: 技能层重试用尽仍失败, 或遇到 6.1 的确定性失败; `replan_count` 达到 `MAX_REPLANS=2` 则置 aborted、终止任务。
- **Planner.replan 一次 LLM 调用** (`max_tokens=1500`, `temperature=0.3`), 输入包括:
 - **system**: 复用主规划的 SYSTEM_PROMPT, 但填入**最新当前位姿与重新生成的通道 B 清单**, 因此 replan 看得到前面步骤新发现的物体;
 - **user (REPLAN_PROMPT)**: 原始任务指令、已成功完成的步骤 `steps[:i]` (**不含当前失败步**)、失败步骤本身、失败原因 (失败结果的 message)、工作记忆感知摘要 (已发现物体名 / "尚未发现目标物体")、任务开始时通道 A 的检索结果、延迟参数规则、记忆使用规则, 以及"可换一种策略绕过失败"的提示。
- **输出**: `{replan_reason, steps[]}`, 其中 steps **只含剩余步骤、从 1 重新编号**, 且同样经过 `_validate_steps` 与 `_validate_resolve_fields` 两道校验。
- **替换方式**: `steps = steps[:i] + new_steps`, 索引 i 保持不变, 下一轮直接执行新计划的第一步; 原失败步结果仍保留在 `results` 中并标记 `replan_after=true`; 同时 `replan_count+1`、工作记忆 `replans+1`。
- **异常处理**: replan 的 LLM 调用或校验本身抛错 → aborted 终止; replan 时若位姿暂时取不到, 降级为 None (prompt 显示"未知") 而不崩溃。

走例: 原计划为 `observe_surroundings → rotate_find_align → approach_aligned`。第 1 步观察成功; 第 2 步 `rotate_find_align` 原地搜索后 repeat 1 次仍报"未找到目标", 触发**第 1 次 replan**: 把已完成的 observe 作为已完成步骤、失败步与失败信息、工作记忆"尚未发现目标物体"、最新位姿和重建的通道 B 清单发给 LLM; LLM 判断应改为主动探索, 返回新剩余步骤 `explore_no_align → approach_aligned`。执行序列被替换为 `observe_surroundings (成功) + explore_no_align + approach_aligned`, 失败的 `rotate_find_align` 记录保留且 `replan_after=true`; 后续两步成功, **任务整体判定成功, replans=1**。若两次 replan 后仍有步骤失败, 则 aborted 终止并判失败。

6.5 最终成败判定

执行循环结束后产出执行报告：

- 正常走完（未 aborted）时，以 **results 最后一步的 success 为准**——因为 replan 可能合法地绕过了失败步骤；aborted 或没有任何结果时才直接判失败；
- 报告字段：**success**、**total_steps**、**completed_steps**（成功步数）、**replans**（重规划次数）、**skill_retries**（技能重试次数）、**results**（每逻辑步最终结果，含 retries / replan_after 标记）、**failed_step**；
- **重规划后最终走完所有剩余步骤即判任务成功**，失败步骤记录仍保留在报告中可追溯；最终成败再交全流程第 6 步的 Reviewer 复核（归因与教训见 Review 环节）。

7 评测

8 统一返回值格式

单步调用

```
{
  "success": true,
  "skill": "physical_look_around",
  "action": "rotate_find_align",
  "message": "对正完成：椅子",
  "data": {
    "perceptions": {
      "objects_found": [
        {
          "name": "椅子",
          "depth": 2.35,
          "confidence": 0.82,
          "direction": 90.0,
          "source_action": "rotate_find_align"
        }
      ],
      "areas_explored": [],
      "final_pose": {"x": 0.0, "y": 0.0, "theta": 90.0}
    }
  }
}
```

长序列任务

```
{
  "success": true,
  "total_steps": 3,
  "completed_steps": 3,
  "replans": 0,
  "skill_retries": 0,
  "failed_step": null,
  "results": [
    {
```

```

        "step": 1,
        "skill": "physical_look_around",
        "action": "observe_surroundings",
        "success": true,
        "message": "观察完成: 8 个方位",
        "elapsed_sec": 45.2,
        "replan_after": false
    }
]
}

```

perceptions 字段说明

每个 action 返回的 `data.perceptions` 包含:

字段	说明
<code>objects_found</code>	发现的物体列表: name, depth, confidence, direction, source_action
<code>areas_explored</code>	探索的区域列表: direction, openness, description
<code>final_pose</code>	执行后机器人位姿: x, y, theta

不同 action 按实际能获取的数据填充, 无数据的字段为空列表。

9 测试

```

# 运行全部测试 (不需要 ROS)
python3 -m pytest tests/ -v

# 或单独运行
python3 tests/test_memory.py      # 记忆系统功能测试
python3 tests/test_replan.py      # 重规划判定逻辑测试
python3 tests/test_json.py        # JSON 解析容错测试
python3 tests/test_motion.py      # 运动参数测试
python3 tests/test_pid.py         # PID 控制器测试
python3 tests/test_planner.py     # 规划器校验测试
python3 tests/test_skills.py      # Skill 注册表测试
python3 tests/test_vlm.py         # VLM 工具层测试
python3 tests/test_direction_mapping.py # 方向映射与坐标变换测试

```

10 后续论文课题方向

10.0 短期方向

需要提前确认memory没问题; 真机调试出demo, skill的补充优化, 记忆系统的优化

10.1 所谓的创新点

自主编排自生长技能库：在数量充足、语义足够原子的基础 action 之上，把反复成功的动作编排 "结晶" 为可复用、可嵌套再组合的高级 skill，后续任务检索复用、失败时迭代进化，使能力随经验复合增长。

自进化记忆：记忆不仅来自每次视觉感知的新增，还能基于已有记忆结合几何 / 空间推理（方位传递、遮挡推断、区域闭合等）主动派生新记忆，实现记忆的自我发育与自我补全，而非只靠逐帧图像累积。

10.2 仿真

10.3 sota

10.4 消融实验

10.5 评测系统